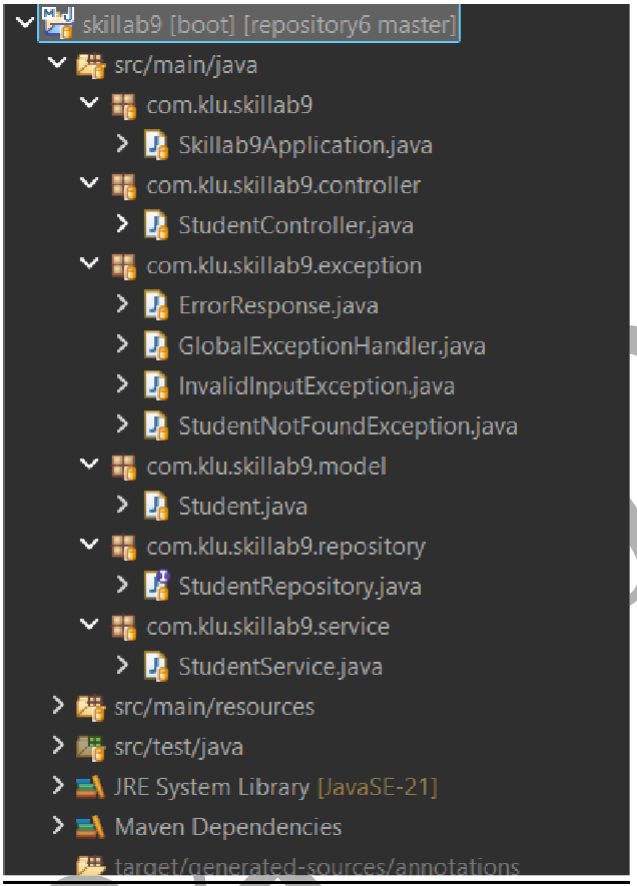
NAME:	PULLURU SATHVIK
ID:	2400032783
SECTION:	S-8
SUBJECT:	FULL STACK APPLICATION DEVELOPMENT

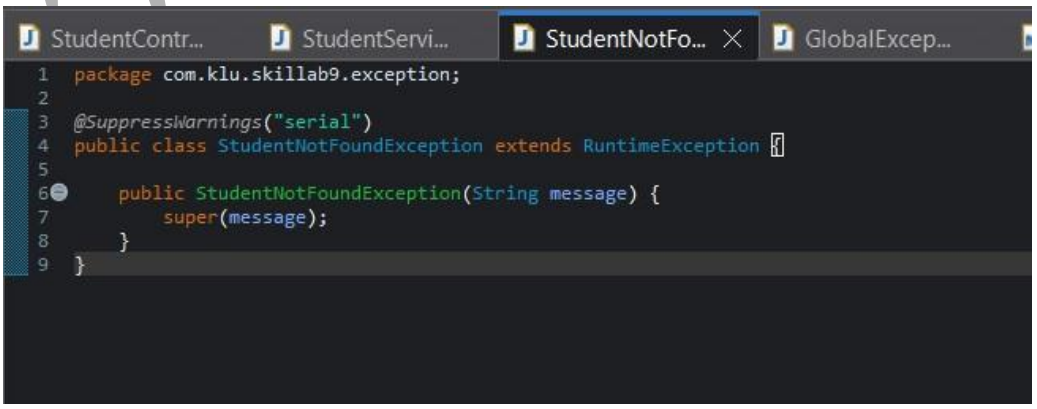
(DEEMED TO BE UNIVERSITY)

#SKILL - 9 Global Exception Handling using @ControllerAdvice*Tasks:*

1. Create a custom exception class named StudentNotFoundException.

Output:-





2400032783

2. Create a REST endpoint /student/{id} that retrieves student information and throws the custom exception when the given student ID is invalid.

Output:-

```
StudentContr...   StudentServi...   StudentNotFo...   ErrorRespons...
package com.klu.skillab9.exception;

import java.time.LocalDateTime;

public class ErrorResponse {

    private LocalDateTime timestamp;
    private String message;
    private int statusCode;

    public ErrorResponse(LocalDateTime timestamp, String message, int statusCode) {
        this.timestamp = timestamp;
        this.message = message;
        this.statusCode = statusCode;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public String getMessage() {
        return message;
    }

    public int getStatusCode() {
        return statusCode;
    }
}
```

3. Create a GlobalExceptionHandler class using @ControllerAdvice to handle exceptions across the entire application.

Output:-

```
StudentContr...   StudentServi...   StudentNotFo...   GlobalExcep...   ErrorRespo
package com.klu.skillab9.exception;

import java.time.LocalDateTime;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleStudentNotFound(StudentNotFoundException ex){

        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(),
                ex.getMessage(),
                HttpStatus.NOT_FOUND.value()
        );

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ErrorResponse> handleInvalidInput(InvalidInputException ex){

        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(),
                ex.getMessage(),
                HttpStatus.BAD_REQUEST.value()
        );

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}
```

4. Add a method with @ExceptionHandler(StudentNotFoundException.class) to return a meaningful error response along with an appropriate HTTP status code.

Output:-

```java
    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

5. Add a second custom exception, such as InvalidInputException, and handle it inside the same @ControllerAdvice class.

Output:-

```java
package com.klu.skillab9.exception;

@SuppressWarnings("serial")
public class InvalidInputException extends RuntimeException {

    public InvalidInputException(String message) {
        super(message);
    }
}
```

6. Return a structured JSON response (fields like timestamp, message, statusCode) from the exception handler to make the error readable and consistent.

Output:-

GET http://localhost:8080/student/100

Body

raw JSON

```
{
  "id": 1,
  "name": "Sathvik",
  "course": "CSE"
}
```

Body 404 Not Found

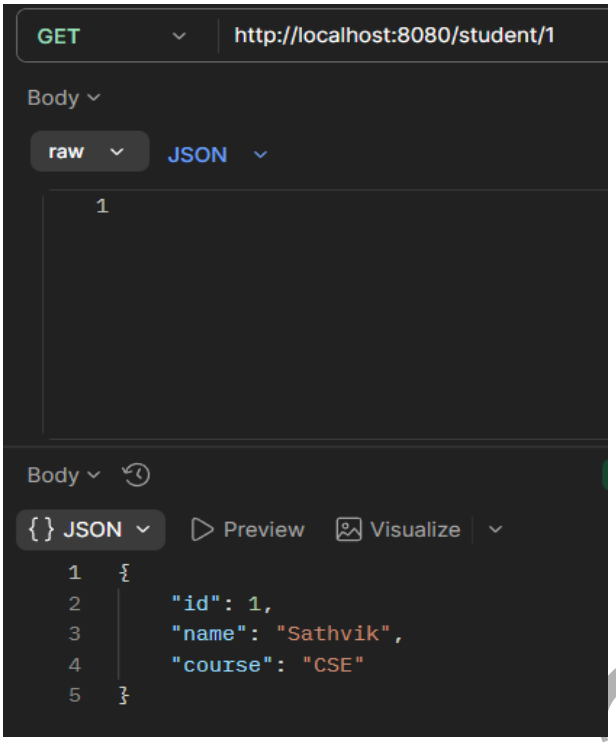
{} JSON Preview Debug with AI

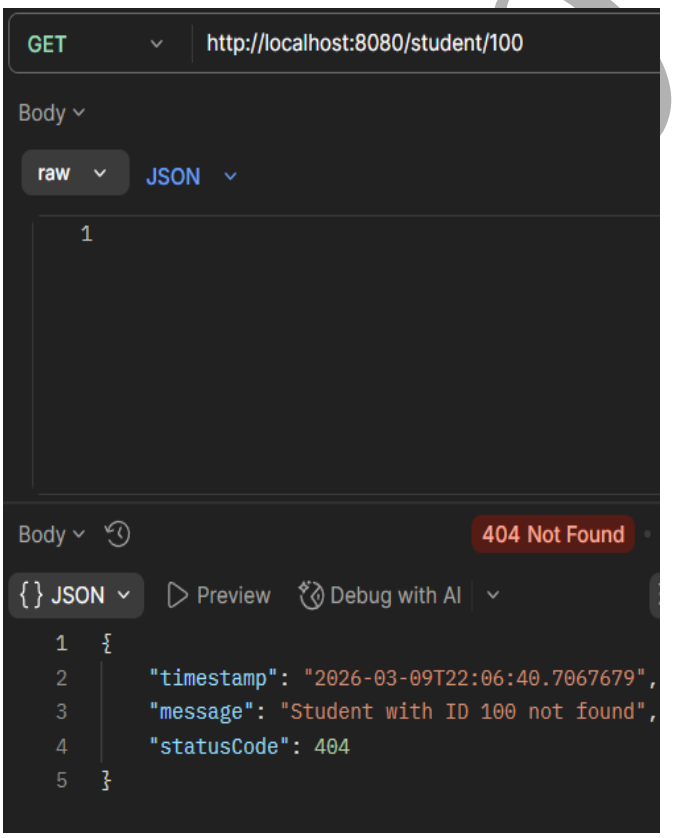
```
{
    "timestamp": "2026-03-09T22:03:33.4239534",
    "message": "Student with ID 100 not found",
    "statusCode": 404
}
```

032783

7. Test the API with valid and invalid student IDs and observe the difference in responses.

<u>Output:-</u>

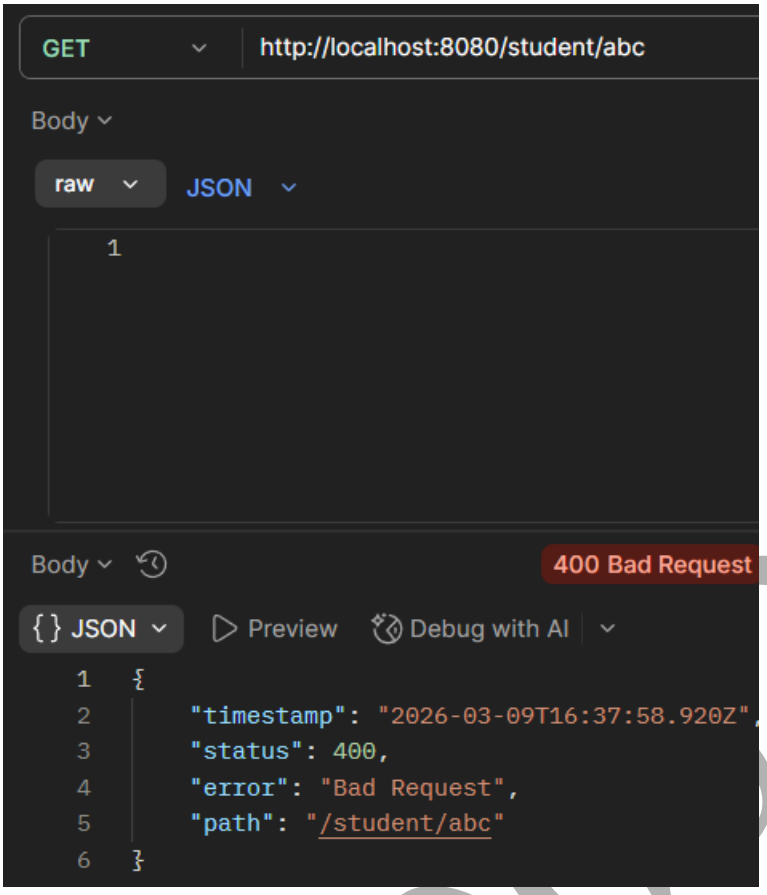




032783

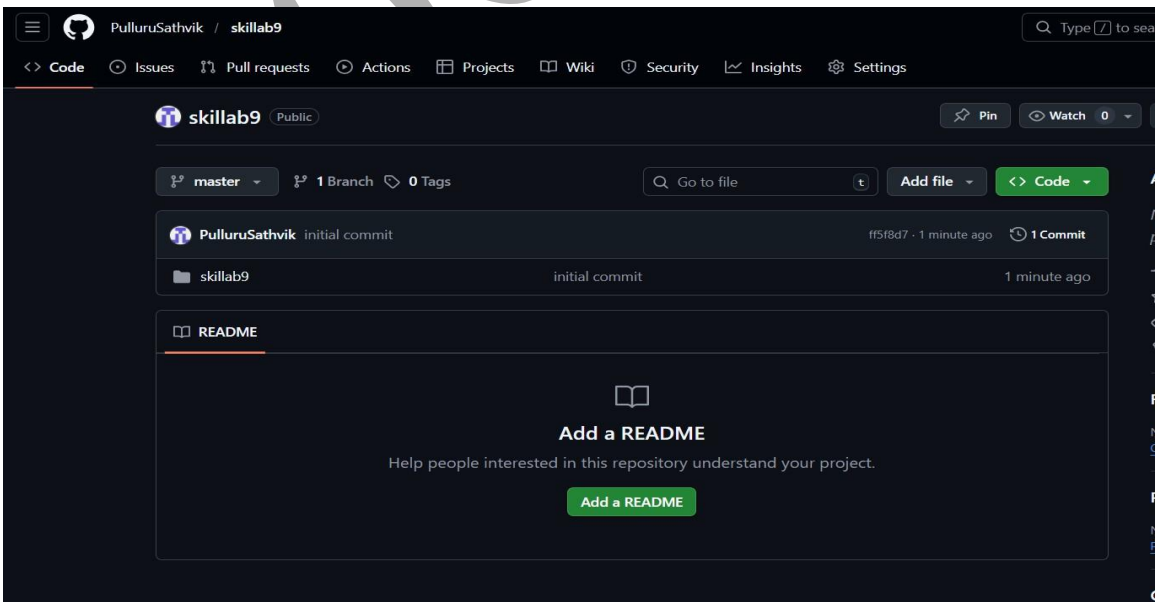
8. Test an endpoint with invalid input format (e.g., sending text instead of a number) to trigger the second custom exception using Postman.

Output:-



9. Push the Spring Boot project to GitHub.

Output:-



GITHUB REPO LINK :- https://github.com/PulluruSathvik/skillab9